

CS-338 Intro to the Theory of Computation

- Lecture #4

➤ **Non Regular Languages**

- Prof Pietro S. Oliveto

Department of Computer Science and Engineering

Southern University of Science and Technology (SUSTech)

- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

18.404/6.840 Lecture 4

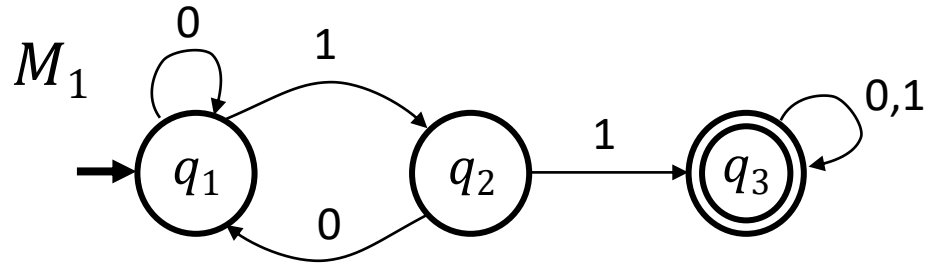
Last time:

- Regular Expressions
- Regular expressions $\rightarrow$ finite automata
- Finite automata $\rightarrow$ regular expressions

Today: (Sipser §1.4)

- Non regular Languages
- Proving languages aren't regular

Finite Automata (Recap)



States: $q_1 q_2 q_3$

Transitions: $\xrightarrow{1}$

Start state: $\rightarrow \bigcirc$

Accept states: $\bigcirc\bigcirc$

Input: finite string

Output: Accept or Reject

Computation process: Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

Examples: 01101 $\rightarrow$ Accept
00101 $\rightarrow$ Reject

M_1 accepts exactly those strings in A where
 $A = \{w \mid w \text{ contains substring } 11\}.$

Say that A is the language of M_1 and that M_1 recognizes A and that $A = L(M_1).$

A language is **regular** if some finite automaton recognises it

Regular Expressions: Formal Definition

Say that R is a *regular expression* if R is

1. a for some a in the alphabet Σ ,
2. ϵ ,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions, or
6. (R_1^*) , where R_1 is a regular expression.

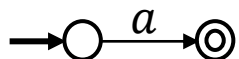
Regular Expressions $\rightarrow$ NFA $\rightarrow$ DFA

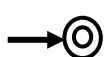
Theorem: If R is a regular expr and $A = L(R)$ then A is regular

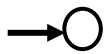
Proof: Convert R to equivalent NFA M :

If R is atomic:

Equivalent M is:

$R = a$ for $a \in \Sigma$ 

$R = \varepsilon$ 

$R = \emptyset$ 

If R is composite:

$R = R_1 \cup R_2$

$R = R_1 \circ R_2$

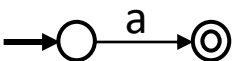
$R = R_1^*$

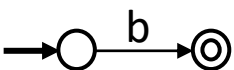


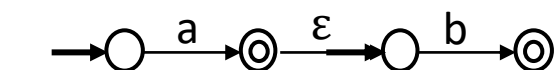
Use closure constructions

Example:

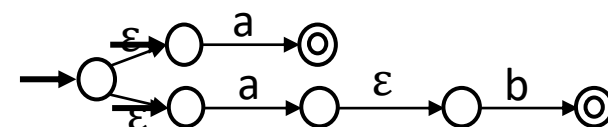
Convert $(a \cup ab)^*$ to equivalent NFA

a : 

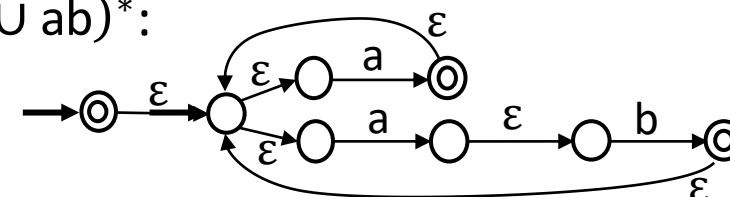
b : 

ab : 

$a \cup ab$:



$(a \cup ab)^*$:



DFA → Special GNFA

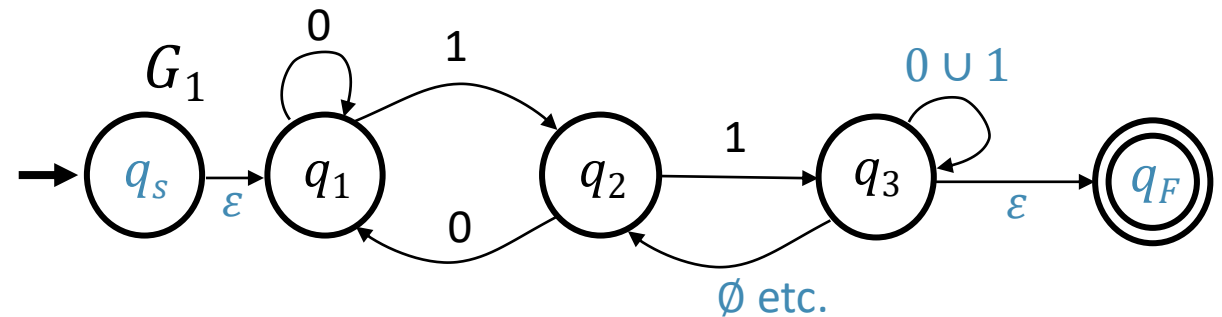
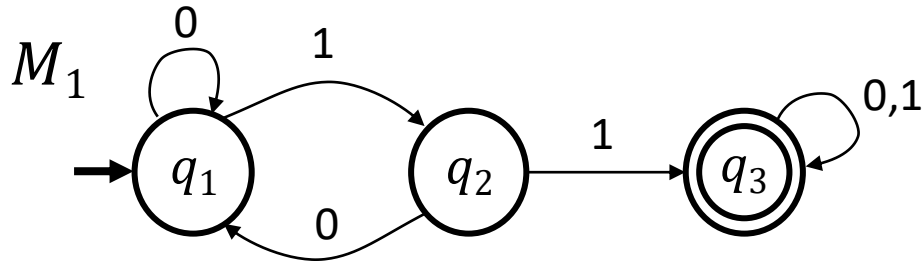
- Add a **new start state** with an ε arrow to the old start state
- Add a **new accept state** with an ε arrow from each accept state
- If any arrows have multiple labels **replace with the union**
- Add arrows labelled $\emptyset$ between states with no arrows

Reminder

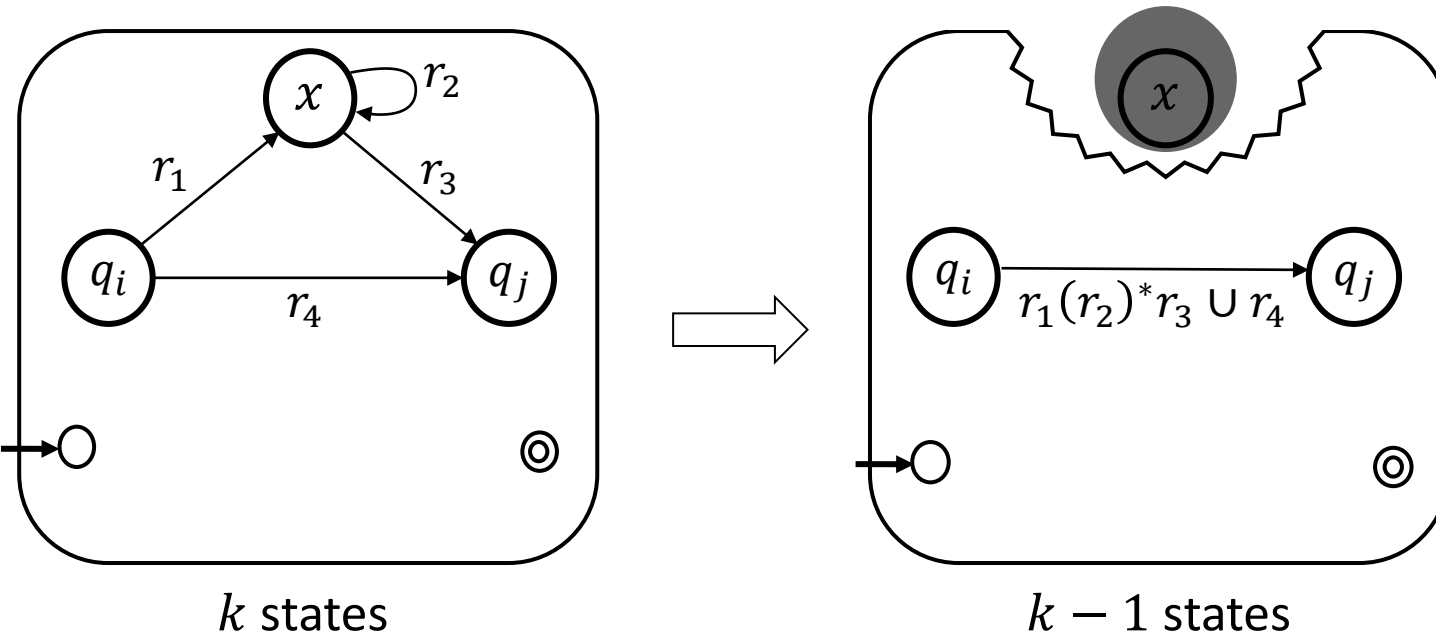
For convenience we will assume:

- One accept state, separate from the start state
- One arrow from each state to each state, except
 - a) only exiting the start state
 - b) only entering the accept state

Example



k -state GNFA $\rightarrow (k-1)$ -state GNFA



1. Pick any state x except the start and accept states.
2. Remove x .
3. Repair the damage by recovering all paths that went through x .
4. Make the indicated change for each pair of states q_i, q_j .

Thus DFAs and regular expressions are equivalent.

Regular or non-regular Languages?

What are the limitations of DFAs?

To understand the power of finite automata you must understand its limitations

Is the following language regular?

Here $\Sigma = \{0,1\}$.

1. 01
2. 0^21^2
3. 0^31^3
4. 0^21^3
5. $0^21^2 \cup 0^31^3$
6. 0^*1^*
7. 0^n1^n for $n \geq 0$

- The machine seems to need to remember how many 0s it has seen as it reads the input
- The number of 0s isn't limited, so it has to be able to track infinite possibilities
- But it cannot do so with a finite number of states!

Non-Regular Languages

How do we show a language is not regular?

- Remember, to show a language *is* regular, we give a DFA.
- To show a language is *not* regular, we must give a proof.
- It is not enough to say that you couldn't find a DFA for it, therefore the language isn't regular.

Two examples: Here $\Sigma = \{0,1\}$.

1. Let $B = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

Intuition: B is not regular because DFAs cannot count unboundedly.

2. Let $C = \{w \mid w \text{ has equal numbers of 01 and 10 substrings}\}$

$$\overline{0101} \notin C \quad \overline{0110} \in C$$

Intuition: C is not regular because DFAs cannot count unboundedly.

However C is regular!

Moral: You need to give a proof.

Method for Proving Non-regularity

The theorem states that **all** regular languages have a special property:

“All strings of a regular language that are long at least as much as the pumping length can be ‘pumped’ any number of times and the result will remain in the language”

Pumping Lemma: For every regular language A , there is a number p (the “pumping length”) such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$
 - 2) $|y| > 0$
 - 3) $|xy| \leq p$
- $y^i = \underbrace{yy \cdots y}_i$

Informally: A is regular $\rightarrow$ every long string in A can be pumped and the result stays in A .

- Either x or z may be ε , but condition 2 says that $y \neq \varepsilon$
- Observe that without Condition 2, the theorem would be trivially true

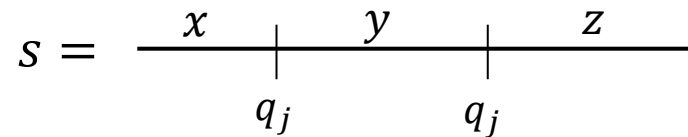
Proof of Pumping Lemma

Pumping Lemma: For every regular language A , there is a number p (the “pumping length”) such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

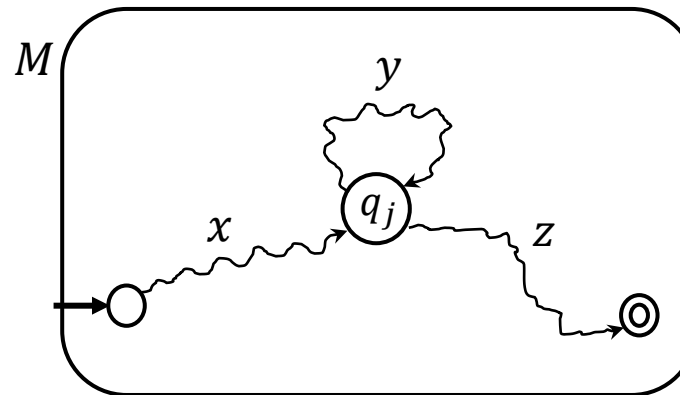
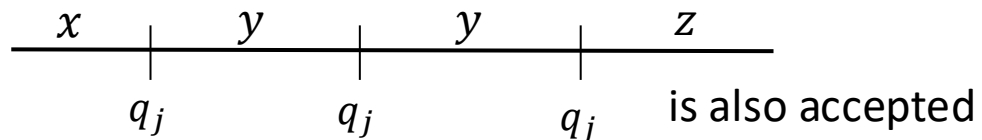
- 1) $xy^iz \in A$ for all $i \geq 0$ $y^i = \underbrace{yy \cdots y}_i$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Informally: A is regular $\rightarrow$ every long string in A can be pumped and the result stays in A .

Proof: Let DFA M recognize A . Let p be the number of states in M . Pick $s \in A$ where $|s| \geq p$.



M will repeat a state q_j when reading s because s is so long.



The path that M follows when reading s .

1. M accepts xyz, xy^2z and xy^iz for all $i \geq 0$
2. $y > 0$ as it was part of s
3. $|xy| \leq p$ because the first $p + 1$ visited states must contain a repetition (i.e., $p = |Q|$)

Using Pumping Lemma to Prove Non-regularity

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$ $y^i = yy \cdots y$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

- Assume $\mathcal{L}$ is regular
- Then by Pumping Lemma there **exists** a pumping length p such that all strings $s \in \mathcal{L} \mid |s| \geq p$ can be pumped
- Find a string $s \in \mathcal{L}$ of length p or greater that cannot be pumped
- Demonstrate that it cannot be pumped by:
 - Considering **all ways** of dividing s into xyz
 - Show that for each of these ways there exists a value of $i \mid i \geq 0$ for which **$xy^iz \notin \mathcal{L}$**
- The existence of $s \in \mathcal{L}$ that cannot be pumped leads to a **contradiction**. Hence $s \notin \mathcal{L}$.

Example 1 of Proving Non-regularity

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$ $y^i = yy \cdots y$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Let $D = \{0^k 1^k \mid k \geq 0\}$

Show: D is not regular

Proof by Contradiction:

Assume (to get a contradiction) that D is regular.

The pumping lemma gives p as above. Let $s = 0^p 1^p \in D$. ($|s| \geq p$ and $s \in A$)

Pumping lemma says that we can divide $s = xyz$ satisfying the 3 conditions.

1. y consists of only 0s $\Rightarrow xy^2z$ has excess 0s and thus $xy^2z \notin D$
2. y consists of only 1s $\Rightarrow$ opposite contradiction
3. y consists of both 0s and 1s $\Rightarrow$ they will be out of order

$$s = \begin{array}{ccccccc} 0 & 0 & 0 & \cdots & 0 & 0 & 0 & 1 & 1 & 1 & \cdots & 1 & 1 & 1 \\ \hline & x & & & y & & & & & & z & & & \\ & \leftarrow & \leq p & \rightarrow & & & & & & & & & & \end{array}$$

All ways of splitting s contradict the pumping lemma. Therefore our assumption (D is regular) is false.

We conclude that D is not regular.

Note: Cases 2. and 3. can be eliminated by considering Condition 3) of the Pumping Lemma.

Example 2 of Proving Non-regularity

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^iz \in A$ for all $i \geq 0$ $y^i = yy \cdots y$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Let $F = \{ww \mid w \in \Sigma^*\}$. Say $\Sigma^* = \{0,1\}$.

Show: F is not regular

Proof by Contradiction:

Assume (for contradiction) that F is regular.

The pumping lemma gives p as above. Need to choose $s \in F$. Which s ?

Try $s = 0^p 0^p \in F$. But that s can be pumped and stay inside F . Bad choice.

Try $s = 0^p 1 0^p 1 \in F$. Show cannot be pumped $s = xyz$ satisfying the 3 conditions. $xyyz \notin F$ Contradiction! Therefore, F is not regular.

$$s = \frac{000 \cdots 000000 \cdots 000}{\begin{array}{ccc} x & | & y & | & z \\ \leftarrow & \leq p & \rightarrow & & \end{array}} \quad y = 00$$

$$s = \frac{000 \cdots 001000 \cdots 001}{\begin{array}{ccc} x & | & y & | & z \\ \leftarrow & \leq p & \rightarrow & & \end{array}}$$

Example 3 of Proving Non-regularity

Variant: Combine closure properties with the Pumping Lemma.

Let $B = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$

Show: B is not regular

Proof by Contradiction:

- Assume (for contradiction) that B is regular.
- We know that 0^*1^* is regular so $B \cap 0^*1^*$ is regular (closure under intersection).
- But $D = \{0^k1^k \mid k \geq 0\} = B \cap 0^*1^*$ and we already showed D is not regular.
Contradiction!
- Therefore our assumption is false, so B is not regular.

Example 4 of Proving Non-regularity

Let $G = \{1^{n^2} \mid n \geq 0\}$

Show: the unary language G is not regular.

G contains all strings whose length is a perfect square: 0,1,4,9,16,25,36,49...

Proof by Contradiction:

- Assume (for contradiction) that D is regular.
- Let p be the pumping length, and let s be the string 1^{p^2} .
- Pumping lemma guarantees s can be split into $xyz \mid xy^iz \in G$ for all $i \geq 0$.

Consider xyz and $xyyz$:

- $|y| \leq p$ by Condition 3 $\Rightarrow |xyz| = p^2$ and $|xyyz| \leq p^2 + p$
But $p^2 + p < p^2 + 2p + 1 = (p + 1)^2$ which is the next element in the language
- Moreover, by Condition 2 $y \notin \epsilon$, thus $|xyyz| > p^2$
- Therefore, $|xyyz|$ lies between two perfect squares and cannot be in the language
- Our assumption is false, so B is not regular.

Example 5 of Proving Non-regularity

Variant: “Pumping down”

Let $E = \{0^i 1^j \mid i > j\}$

Show: E is not regular

Proof by Contradiction:

- Assume (for contradiction) that E is regular.
- Let p be the pumping length, and let s be the string $0^{p+1}1^p$.
- Pumping lemma guarantees s can be split into $xyz \mid xy^i z \in E$ for all $i \geq 0$.
- By Condition 3 y consists only of 0s
- E contains strings 0^*1^* that contain more 0s than 1s, so adding more copies of y doesn't help $\Rightarrow$ “Pump down”
- Consider $i = 0$! xy^0z should also be in the language: $xz \notin E$ because s has only one more 0 than 1s.
- Contradiction! Therefore our assumption is false, so E is not regular.

Pumping Lemma: For every regular language A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

- 1) $xy^i z \in A$ for all $i \geq 0$ $y^i = yy \cdots y$
- 2) $y \neq \varepsilon$
- 3) $|xy| \leq p$

Quick review of today

1. Proving languages not regular by using the pumping lemma and closure properties